

Wonder Ticketing — Production-Ready Technical Specification

Version: 1.0

Date: 2026-05-29

Prepared for: Wonder Payment Internal Development Team

Classification: Internal — Confidential

Table of Contents

- [1. System Overview](#)
 - [2. Technology Stack](#)
 - [3. System Architecture](#)
 - [4. Database Design](#)
 - [5. API Specification](#)
 - [6. Frontend Component Architecture](#)
 - [7. Payment Flow \(Wonder Payment Integration\)](#)
 - [8. Free/Pro Feature Gating Logic](#)
 - [9. Responsive Design Strategy](#)
 - [10. Authentication & Authorization](#)
 - [11. Notification System](#)
 - [12. Deployment & Infrastructure](#)
 - [13. Testing Strategy](#)
 - [14. Development Phases & Sprint Plan](#)
 - [15. Appendix: Flow Diagrams](#)
-

1. System Overview

1.1 Product Definition

Wonder Ticketing 是由 Wonder Payment（持牌支付公司）推出的自助活動售票 SaaS 平台，專為香港中小型活動主辦方（Event Organizer）而設。平台採用 **SaaS 多租戶模式**，任何已登入用戶均可自行建立及管理活動，無需管理員審批。

1.2 Core Business Rules

規則	說明
多租戶隔離	每位主辦方只能查看及管理自己建立的活動、訂單、票券
Free/Pro 雙軌制	Free 用戶收取 5% 全包費率；Pro 用戶月費 HK\$460 + 3% 全包費率
支付閉環	所有交易經由 Wonder Payment 處理，平台費已含支付處理費
活動規模	每場活動最多 300 人（可擴展）
即時結算	T+1 結算至主辦方帳戶
雙語支援	介面及活動內容支援繁體中文 / English

1.3 User Roles

角色	權限
Attendee（買家）	瀏覽活動、購票、查看票券、推薦朋友
Organizer（主辦方）	建立/管理活動、管理票種、查看訂單、驗票、查看報表
Admin（平台管理員）	全平台管理、精選活動推薦、用戶管理、系統設定

2. Technology Stack

2.1 Core Stack

Layer	Technology	Version	Justification
Frontend	React	19	最新 concurrent features、Server Components ready

Styling	Tailwind CSS	4	Utility-first、零 runtime overhead、OKLCH 色彩
Routing	wouter	3.7	輕量 (<2KB)、hooks-first API
Backend	Express	4	成熟穩定、middleware 生態豐富
API Layer	tRPC	11	End-to-end type safety、零 code generation
ORM	Drizzle	latest	Type-safe SQL、輕量、migration-first
Database	MySQL (TiDB)	8.0 compat	分散式、水平擴展、MySQL 相容
Runtime	Node.js	22 LTS	長期支援、native ESM
Package Manager	pnpm	latest	快速、磁碟高效
Build Tool	Vite	latest	極速 HMR、optimized production build

2.2 Key Libraries

Category	Library	Purpose
UI Components	shadcn/ui + Radix	Accessible、可客製化 headless components
Validation	Zod	Runtime schema validation (前後端共用)
QR Code	qrcode	票券 QR Code 生成
Icons	Lucide React	一致的 icon set
Date	Native Date + Intl	時區處理、本地化格式
Charts	Chart.js / Recharts	Pro 版銷售分析圖表
i18n	react-i18next	中英雙語切換
Camera	html5-qrcode	手機掃描 QR Code

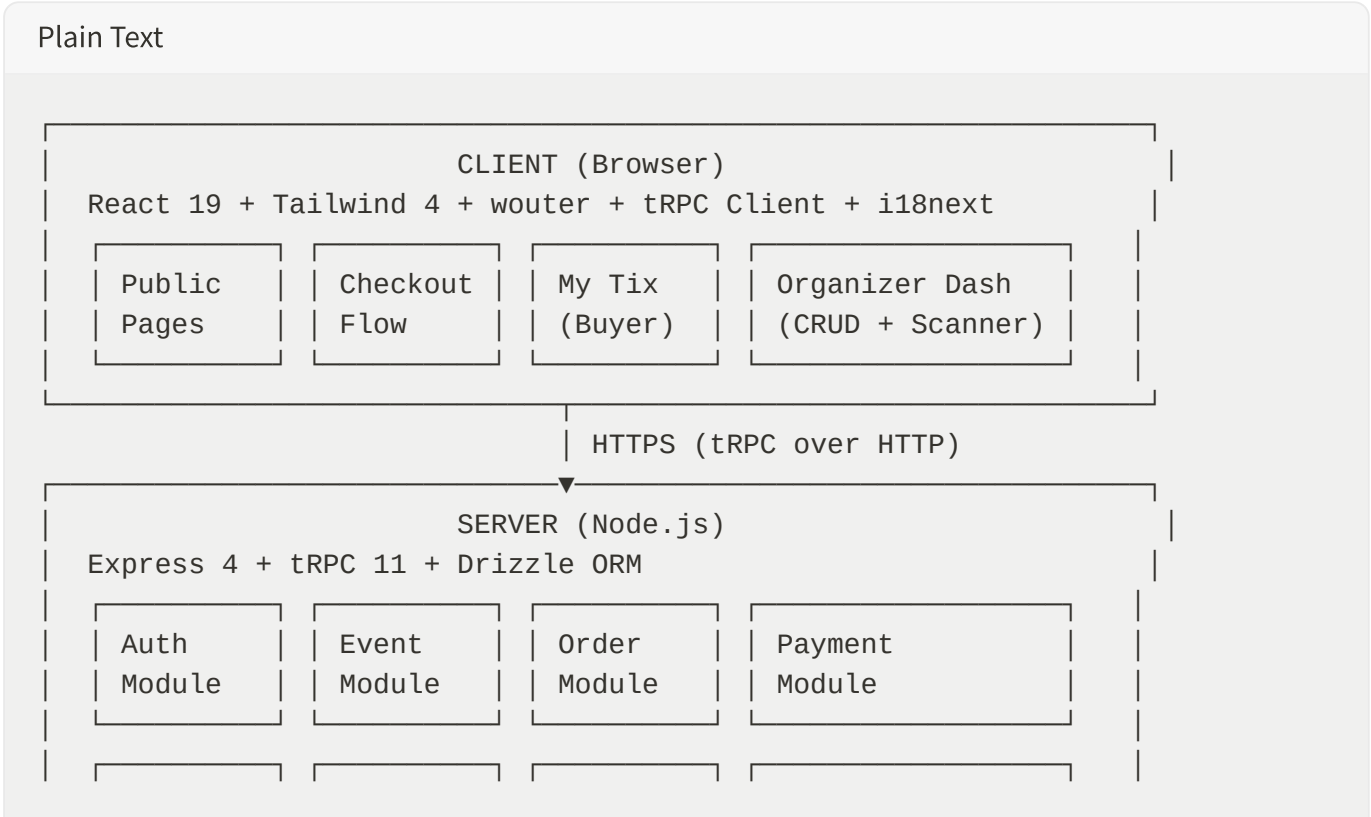
Storage	S3 (Built-in)	Banner 圖片、附件儲存
---------	---------------	----------------

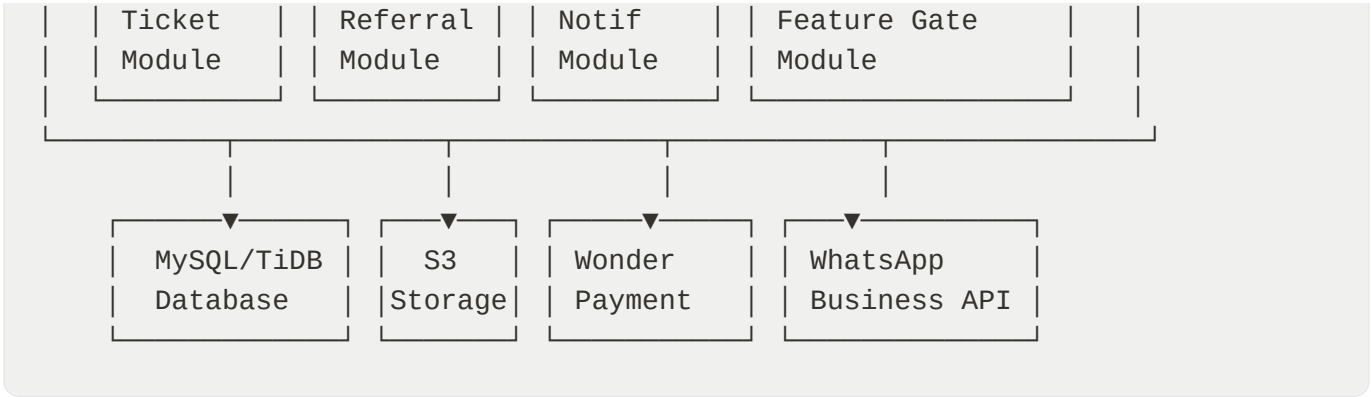
2.3 External Services

Service	Provider	Purpose
Payment Gateway	Wonder Payment API	FPS/PayMe/WeChat/Alipay/信用卡
Email	Forge Notification API	訂單確認、活動提醒
WhatsApp	WhatsApp Business API	票券通知、自動客服
Authentication	Manus OAuth	用戶登入/註冊
Object Storage	S3 (Built-in)	圖片、文件儲存
LLM	Built-in LLM API	AI 活動文案生成 (Pro)

3. System Architecture

3.1 High-Level Architecture





3.2 Request Flow

Plain Text

```
Client → tRPC Client → HTTP POST /api/trpc/{procedure}
  → Express middleware (CORS, cookies, auth context)
  → tRPC Router → Procedure (public/protected/admin)
    → Zod Input Validation
      → Business Logic (db.ts helpers)
        → Drizzle ORM → MySQL
        → Response (Superjson serialized)
      ← tRPC Response
    ← HTTP Response
  ← React Query cache update
  ← UI re-render
```

3.3 Directory Structure

Plain Text

```
wonder-ticketing/
├── client/
│   ├── index.html           # Entry HTML
│   ├── public/              # Static config files only
│   └── src/
│       ├── App.tsx          # Route definitions
│       ├── main.tsx         # Provider composition
│       ├── index.css        # Global theme (OKLCH tokens)
│       └── lib/
│           ├── trpc.ts       # tRPC client binding
│           ├── utils.ts      # cn() helper
│           └── i18n.ts       # i18next configuration
│       ├── locales/
│           ├── zh-TW.json    # 繁體中文翻譯
│           └── en.json       # English translations
│       └── contexts/
```

- └─ ThemeContext.tsx # Light/dark mode
 - └─ LanguageContext.tsx # i18n context
 - └─ hooks/
 - └─ useFeatureGate.ts # Free/Pro feature check
 - └─ useMobile.tsx # Responsive breakpoint hook
 - └─ components/
 - └─ ui/ # shadcn/ui primitives
 - └─ LanguageSwitcher.tsx # 中/EN toggle
 - └─ FeatureGate.tsx # Conditional render by plan
 - └─ PullToRefresh.tsx # Mobile gesture
 - └─ SeatMap/ # Seat selection components
 - └─ pages/
 - └─ Home.tsx # Landing + event discovery
 - └─ EventPage.tsx # Public event detail
 - └─ Checkout.tsx # Payment flow
 - └─ OrderConfirmation.tsx # Post-payment
 - └─ MyTickets.tsx # Buyer ticket wallet
 - └─ Explore.tsx # Event discovery/search
 - └─ admin/
 - └─ AdminLayout.tsx # Dashboard shell
 - └─ Dashboard.tsx # Stats overview
 - └─ Events.tsx # Event list
 - └─ EventForm.tsx # Create/edit event
 - └─ Orders.tsx # Order management
 - └─ CheckIn.tsx # QR scanner
 - └─ Analytics.tsx # Pro: sales reports
 - └─ CRM.tsx # Pro: attendee CRM
 - └─ Referrals.tsx # Referral management
 - └─ Settings.tsx # Organizer settings
- └─ server/
 - └─ _core/ # Framework (DO NOT EDIT)
 - └─ index.ts # Server entry
 - └─ context.ts # Request context builder
 - └─ trpc.ts # Procedure definitions
 - └─ oauth.ts # Manus OAuth
 - └─ env.ts # Environment variables
 - └─ llm.ts # AI text generation
 - └─ notification.ts # Push notification
 - └─ imageGeneration.ts # Image AI
 - └─ routers/
 - └─ event.ts # Event CRUD procedures
 - └─ ticketType.ts # Ticket type management
 - └─ order.ts # Order/checkout procedures
 - └─ ticket.ts # Ticket/check-in procedures
 - └─ referral.ts # Referral code procedures
 - └─ donation.ts # Donation procedures
 - └─ discount.ts # Discount code procedures
 - └─ waitlist.ts # Waitlist procedures

```

| | | | analytics.ts          # Pro: reporting procedures
| | | | | crm.ts              # Pro: CRM procedures
| | | | | └─ subscription.ts  # Plan management
| | | | db.ts                 # Database query helpers
| | | | | payment.ts          # Wonder Payment integration
| | | | | email.ts            # Email + QR generation
| | | | | whatsapp.ts         # WhatsApp Business API
| | | | | featureGate.ts      # Server-side plan check
| | | | | storage.ts          # S3 helpers
| | | | | └─ *.test.ts        # Vitest test files
| | └─ drizzle/
| | | | schema.ts             # All table definitions
| | | | | relations.ts        # Table relations
| | | | | └─ migrations/      # SQL migration files
| | └─ shared/
| | | | const.ts              # Shared constants
| | | | | types.ts            # Shared TypeScript types
| | | | | └─ features.ts      # Feature flag definitions
| └─ package.json

```

4. Database Design

4.1 Entity Relationship Overview

Plain Text

```

users —1:N→ events —1:N→ ticket_types
      |
      | ─1:N→ orders —1:N→ order_items
      |           |
      |           └─1:N→ tickets
      |
      ─1:N→ donations
      ─1:N→ discount_codes
      ─1:N→ waitlist_entries
      ─1:N→ event_seats

users —1:1→ subscriptions
users —1:N→ referral_codes
users —1:N→ referral_uses

```

4.2 Table Definitions

4.2.1 users — 用戶表

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	用戶 ID
openId	VARCHAR(64)	UNIQUE, NOT NULL	OAuth 唯一標識
name	TEXT		顯示名稱
email	VARCHAR(320)		Email 地址
phone	VARCHAR(32)		電話號碼
loginMethod	VARCHAR(64)		登入方式
role	ENUM('user','admin')	DEFAULT 'user'	系統角色
plan	ENUM('free','pro')	DEFAULT 'free'	訂閱方案
planExpiresAt	TIMESTAMP	NULLABLE	Pro 到期時間
locale	VARCHAR(5)	DEFAULT 'zh-TW'	偏好語言
referralCode	VARCHAR(16)	UNIQUE	個人推薦碼
referredBy	INT	FK → users.id	推薦人
createdAt	TIMESTAMP	DEFAULT NOW()	建立時間
updatedAt	TIMESTAMP	ON UPDATE NOW()	更新時間
lastSignedIn	TIMESTAMP	DEFAULT NOW()	最後登入

4.2.2 subscriptions — 訂閱表

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	訂閱 ID
userId	INT	FK → users.id, UNIQUE	用戶 ID
plan	ENUM('free','pro')	NOT NULL	方案類型
status	ENUM('active','cancelled','expired','trial')	NOT NULL	狀態
currentPeriodStart	TIMESTAMP		當前週期開始
currentPeriodEnd	TIMESTAMP		當前週期結束

cancelAtPeriodEnd	BOOLEAN	DEFAULT FALSE	週期結束後取消
wonderPaymentSubscriptionId	VARCHAR(255)		Wonder Payment 訂閱 ID
createdAt	TIMESTAMP	DEFAULT NOW()	
updatedAt	TIMESTAMP	ON UPDATE NOW()	

4.2.3 events — 活動表

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	活動 ID
title	VARCHAR(255)	NOT NULL	活動名稱
titleEn	VARCHAR(255)		英文名稱
description	TEXT		活動描述（中文）
descriptionEn	TEXT		活動描述（英文）
bannerUrl	TEXT		Banner 圖片 URL
eventDate	TIMESTAMP	NOT NULL	活動開始時間
eventEndDate	TIMESTAMP		活動結束時間
venue	VARCHAR(500)	NOT NULL	場地名稱
venueAddress	TEXT		場地地址
venueLatLng	VARCHAR(64)		經緯度（地圖顯示）
organizerName	VARCHAR(255)		主辦方名稱
organizerDescription	TEXT		主辦方簡介
termsAndConditions	TEXT		條款及細則
maxAttendees	INT	DEFAULT 300	最大參加人數
category	VARCHAR(64)		活動分類標籤
eventType	ENUM('in_person','online','hybrid')	DEFAULT 'in_person'	活動類型
visibility	ENUM('public','private','members_only')	DEFAULT 'public'	可見性

status	ENUM('draft','published','cancelled','completed')	DEFAULT 'draft'	狀態
slug	VARCHAR(255)	UNIQUE, NOT NULL	URL slug
customUrl	VARCHAR(255)	UNIQUE	Pro: 自訂 URL
enableDonation	BOOLEAN	DEFAULT FALSE	啟用捐款
donationGoal	DECIMAL(10,2)		捐款目標金額
enableSeating	BOOLEAN	DEFAULT FALSE	Pro: 啟用座位圖
seatMapData	JSON		座位圖配置
enablePeerReferral	BOOLEAN	DEFAULT FALSE	Pro: 啟用買家推薦
peerReferralReward	JSON		推薦獎勵設定
createdBy	INT	FK → users.id, NOT NULL	建立者
createdAt	TIMESTAMP	DEFAULT NOW()	
updatedAt	TIMESTAMP	ON UPDATE NOW()	

4.2.4 ticket_types — 票種表

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	票種 ID
eventId	INT	FK → events.id, NOT NULL	所屬活動
name	VARCHAR(255)	NOT NULL	票種名稱
nameEn	VARCHAR(255)		英文名稱
description	TEXT		票種描述
price	DECIMAL(10,2)	NOT NULL	票價
currency	VARCHAR(3)	DEFAULT 'HKD'	幣種
quantity	INT	NOT NULL	總數量
sold	INT	DEFAULT 0	已售數量

sortOrder	INT	DEFAULT 0	排序
salesStart	TIMESTAMP		開售時間
salesEnd	TIMESTAMP		截止時間
minPerOrder	INT	DEFAULT 1	每單最少購買
maxPerOrder	INT	DEFAULT 10	每單最多購買
requireApproval	BOOLEAN	DEFAULT FALSE	需審批
unlockCode	VARCHAR(64)		解鎖碼（隱藏票種）
seatCategory	VARCHAR(64)		座位區域（Pro）
status	ENUM('active','inactive')	DEFAULT 'active'	狀態
createdAt	TIMESTAMP	DEFAULT NOW()	
updatedAt	TIMESTAMP	ON UPDATE NOW()	

4.2.5 orders — 訂單表

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	訂單 ID
orderNumber	VARCHAR(32)	UNIQUE, NOT NULL	訂單編號 (WDR-XXXXXXXXXX)
eventId	INT	FK → events.id, NOT NULL	所屬活動
userId	INT	FK → users.id	買家用戶 ID
buyerName	VARCHAR(255)	NOT NULL	買家姓名
buyerEmail	VARCHAR(320)	NOT NULL	買家 Email
buyerPhone	VARCHAR(32)	NOT NULL	買家電話
totalAmount	DECIMAL(10,2)	NOT NULL	訂單總額
donationAmount	DECIMAL(10,2)	DEFAULT 0	捐款金額
discountAmount	DECIMAL(10,2)	DEFAULT 0	折扣金額
platformFee	DECIMAL(10,2)		平台費（5% or 3%）

currency	VARCHAR(3)	DEFAULT 'HKD'	幣種
paymentMethod	VARCHAR(32)		支付方式 (fps/payme/wechat/ali pay/card)
wonderPaymentId	VARCHAR(255)		Wonder Payment 交易 ID
wonderPaymentStatus	VARCHAR(32)		Wonder Payment 狀態
status	ENUM('pending','paid','cancelled','refunded','partial_refund')	DEFAULT 'pending'	訂單狀態
paidAt	TIMESTAMP		付款時間
refundedAt	TIMESTAMP		退款時間
discountCodeId	INT	FK → discount_codes.id	使用的折扣碼
referralCodeUsed	VARCHAR(32)		使用的推薦碼
createdAt	TIMESTAMP	DEFAULT NOW()	
updatedAt	TIMESTAMP	ON UPDATE NOW()	

4.2.6 order_items — 訂單明細表

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	
orderId	INT	FK → orders.id, NOT NULL	所屬訂單
ticketTypeId	INT	FK → ticket_types.id, NOT NULL	票種
ticketTypeName	VARCHAR(255)	NOT NULL	票種名稱快照
quantity	INT	NOT NULL	數量
unitPrice	DECIMAL(10,2)	NOT NULL	單價
seatNumbers	JSON		座位號碼列表 (Pro)
createdAt	TIMESTAMP	DEFAULT NOW()	

4.2.7 tickets — 票券表

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	
ticketCode	VARCHAR(64)	UNIQUE, NOT NULL	QR Code 內容
orderId	INT	FK → orders.id, NOT NULL	所屬訂單
orderId	INT	FK → order_items.id, NOT NULL	所屬訂單明細
eventId	INT	FK → events.id, NOT NULL	所屬活動
ticketTypeId	INT	FK → ticket_types.id, NOT NULL	票種
ticketTypeName	VARCHAR(255)	NOT NULL	票種名稱
holderName	VARCHAR(255)		持票人姓名
holderEmail	VARCHAR(320)		持票人 Email
holderPhone	VARCHAR(32)		持票人電話
seatNumber	VARCHAR(32)		座位號 (Pro)
status	ENUM('valid','used','cancelled','transferred')	DEFAULT 'valid'	狀態
transferredTo	INT	FK → users.id	轉讓對象
checkedInAt	TIMESTAMP		入場時間
createdAt	TIMESTAMP	DEFAULT NOW()	

4.2.8 donations — 捐款表

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	
eventId	INT	FK → events.id, NOT NULL	所屬活動

orderId	INT	FK → orders.id	關聯訂單
donorName	VARCHAR(255)		捐款人
donorEmail	VARCHAR(320)		Email
amount	DECIMAL(10,2)	NOT NULL	捐款金額
currency	VARCHAR(3)	DEFAULT 'HKD'	
message	TEXT		留言
isAnonymous	BOOLEAN	DEFAULT FALSE	匿名
wonderPaymentId	VARCHAR(255)		交易 ID
status	ENUM('pending','completed','refunded')	DEFAULT 'pending'	
createdAt	TIMESTAMP	DEFAULT NOW()	

4.2.9 discount_codes — 折扣碼表

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	
eventId	INT	FK → events.id, NOT NULL	所屬活動
code	VARCHAR(32)	NOT NULL	折扣碼
type	ENUM('percentage','fixed')	NOT NULL	折扣類型
value	DECIMAL(10,2)	NOT NULL	折扣值
maxUses	INT		最大使用次數
usedCount	INT	DEFAULT 0	已使用次數
minOrderAmount	DECIMAL(10,2)		最低訂單金額
validFrom	TIMESTAMP		生效時間
validUntil	TIMESTAMP		失效時間
applicableTicketTypes	JSON		適用票種 ID 列表

status	ENUM('active','inactive','expired')	DEFAULT 'active'	
createdAt	TIMESTAMP	DEFAULT NOW()	

4.2.10 waitlist_entries — 候補名單表

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	
eventId	INT	FK → events.id, NOT NULL	所屬活動
ticketTypeId	INT	FK → ticket_types.id	指定票種
name	VARCHAR(255)	NOT NULL	姓名
email	VARCHAR(320)	NOT NULL	Email
phone	VARCHAR(32)		電話
quantity	INT	DEFAULT 1	需求數量
status	ENUM('waiting','notified','converted','expired')	DEFAULT 'waiting'	
notifiedAt	TIMESTAMP		通知時間
createdAt	TIMESTAMP	DEFAULT NOW()	

4.2.11 event_seats — 座位表 (Pro)

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	
eventId	INT	FK → events.id, NOT NULL	所屬活動
seatNumber	VARCHAR(32)	NOT NULL	座位編號 (e.g., A-12)
category	VARCHAR(64)		區域分類
row	VARCHAR(8)		排

col	INT		列
price	DECIMAL(10,2)		座位價格（覆蓋票種價格）
status	ENUM('available','reserved','sold','blocked')	DEFAULT 'available'	
ticketId	INT	FK → tickets.id	關聯票券
createdAt	TIMESTAMP	DEFAULT NOW()	

4.2.12 referral_codes — 推薦碼表

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	
userId	INT	FK → users.id, NOT NULL	擁有者
code	VARCHAR(16)	UNIQUE, NOT NULL	推薦碼
type	ENUM('organizer','peer')	NOT NULL	類型
eventId	INT	FK → events.id	Peer Referral 所屬活動
rewardType	ENUM('pro_month','discount','free_ticket','cashback')		獎勵類型
rewardValue	DECIMAL(10,2)		獎勵值
maxUses	INT	DEFAULT 12	最大使用次數
usedCount	INT	DEFAULT 0	已使用次數
status	ENUM('active','inactive','maxed')	DEFAULT 'active'	
createdAt	TIMESTAMP	DEFAULT NOW()	

4.2.13 referral_uses — 推薦使用記錄表

Column	Type	Constraints	Description
--------	------	-------------	-------------

id	INT	PK, AUTO_INCREMENT	
referralCodeId	INT	FK → referral_codes.id, NOT NULL	推薦碼
referrerId	INT	FK → users.id, NOT NULL	推薦人
refereeId	INT	FK → users.id, NOT NULL	被推薦人
rewardGranted	BOOLEAN	DEFAULT FALSE	獎勵已發放
createdAt	TIMESTAMP	DEFAULT NOW()	

4.2.14 custom_questions — 自訂報名問題表

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	
eventId	INT	FK → events.id, NOT NULL	所屬活動
question	VARCHAR(500)	NOT NULL	問題內容
questionEn	VARCHAR(500)		英文問題
type	ENUM('text','select','checkbox','radio')	DEFAULT 'text'	問題類型
options	JSON		選項列表
isRequired	BOOLEAN	DEFAULT FALSE	是否必填
sortOrder	INT	DEFAULT 0	排序

4.2.15 question_responses — 問題回答表

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	
orderId	INT	FK → orders.id, NOT NULL	所屬訂單

questionId	INT	FK → custom_questions.id, NOT NULL	問題
answer	TEXT	NOT NULL	回答內容
createdAt	TIMESTAMP	DEFAULT NOW()	

4.2.16 event_reminders — 活動提醒表

Column	Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	
eventId	INT	FK → events.id, NOT NULL	所屬活動
type	ENUM('email','whatsapp','push')	NOT NULL	通知類型
triggerBefore	INT	NOT NULL	活動前 N 分鐘
templateContent	TEXT		自訂內容
status	ENUM('pending','sent','failed')	DEFAULT 'pending'	
sentAt	TIMESTAMP		發送時間
createdAt	TIMESTAMP	DEFAULT NOW()	

4.3 Indexes Strategy

SQL

```
-- Performance-critical indexes
CREATE INDEX idx_events_createdBy ON events(createdBy);
CREATE INDEX idx_events_status_date ON events(status, eventDate);
CREATE INDEX idx_events_slug ON events(slug);
CREATE INDEX idx_events_category ON events(category);
CREATE INDEX idx_ticket_types_eventId ON ticket_types(eventId);
CREATE INDEX idx_orders_eventId ON orders(eventId);
CREATE INDEX idx_orders_userId ON orders(userId);
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_orders_orderNumber ON orders(orderNumber);
```

```
CREATE INDEX idx_tickets_orderId ON tickets(orderId);
CREATE INDEX idx_tickets_eventId ON tickets(eventId);
CREATE INDEX idx_tickets_ticketCode ON tickets(ticketCode);
CREATE INDEX idx_waitlist_eventId ON waitlist_entries(eventId, status);
CREATE INDEX idx_discount_eventId ON discount_codes(eventId, code);
CREATE INDEX idx_seats_eventId ON event_seats(eventId, status);
CREATE INDEX idx_referral_code ON referral_codes(code);
CREATE INDEX idx_referral_userId ON referral_codes(userId);
```

5. API Specification

5.1 API Architecture

所有 API 通過 tRPC 提供，路徑為 `/api/trpc/{router}.{procedure}`。使用 Superjson 序列化（Date 等複雜類型自動處理）。

Procedure Types:

Type	Auth Required	Use Case
publicProcedure	✗	活動瀏覽、購票、訂單查詢
protectedProcedure	✓	主辦方 CRUD、買家帳戶
adminProcedure	✓ + role=admin	平台管理
proProcedure	✓ + plan=pro	Pro 專屬功能

5.2 Event Router (`trpc.event.*`)

Procedure	Type	Input	Output	Description
listPublished	query/public	{ page?, limit?, category?, search? }	Event[]	公開活動列表
getBySlug	query/public	{ slug: string }	Event null	活動詳情頁
getTicketTypes	query/public	{ eventId: number }	TicketType[]	活動票種
listMine	query/protected	—	Event[]	我的活動列表
getById	query/protected	{ id: number }	Event null	管理用活動詳情

create	mutation/protected	CreateEventInput	{ id, slug }	建立活動
update	mutation/protected	UpdateEventInput	{ success }	更新活動
delete	mutation/protected	{ id: number }	{ success }	刪除活動
duplicate	mutation/protected	{ id: number }	{ id, slug }	複製活動
uploadBanner	mutation/protected	{ fileName, fileData, contentType }	{ url }	上傳 Banner
cancel	mutation/protected	{ id: number, refundAll?: boolean }	{ success }	取消活動+退款

5.3 Order Router (trpc.order.*)

Procedure	Type	Input	Output	Description
checkout	mutation/public	CheckoutInput	{ paymentUrl, orderNumber }	建立訂單+支付
getByNumber	query/public	{ orderNumber }	OrderDetail null	訂單確認頁
listMine	query/protected	{ eventId? }	Order[]	主辦方訂單列表
getDetail	query/protected	{ id: number }	OrderDetail null	訂單詳情
refund	mutation/protected	{ orderId, amount?, reason? }	{ success }	退款
exportCSV	query/protected	{ eventId: number }	{ csvUrl }	匯出參加者名單

CheckoutInput Schema:

TypeScript

```
const CheckoutInput = z.object({
  eventId: z.number(),
  buyerName: z.string().min(1),
  buyerEmail: z.string().email(),
  buyerPhone: z.string().min(1),
})
```

```

items: z.array(z.object({
  ticketTypeId: z.number(),
  quantity: z.number().min(1),
  attendees: z.array(z.object({ // 團體購票
    name: z.string(),
    email: z.string().email().optional(),
    phone: z.string().optional(),
  })).optional(),
  seatNumbers: z.array(z.string()).optional(), // Pro: 座位選擇
})),
donationAmount: z.number().min(0).optional(),
discountCode: z.string().optional(),
referralCode: z.string().optional(),
customAnswers: z.array(z.object({
  questionId: z.number(),
  answer: z.string(),
})).optional(),
paymentMethod: z.enum(['fps', 'payme', 'wechat', 'alipay', 'card']),
origin: z.string(),
locale: z.enum(['zh-TW', 'en']).default('zh-TW'),
});

```

5.4 Ticket Router (trpc.ticket.*)

Procedure	Type	Input	Output	Description
getByCode	query/public	{ ticketCode }	TicketDetail null	票券查詢
generateQR	query/public	{ ticketCode }	{ qrDataUrl }	生成 QR Code
checkIn	mutation/protected	{ ticketCode }	{ ticketInfo, event }	驗票入場
listByEvent	query/protected	{ eventId }	Ticket[]	活動票券列表
transfer	mutation/protected	{ ticketId, toEmail }	{ success }	票券轉讓
myTickets	query/protected	—	Ticket[]	買家我的票券

5.5 Referral Router (trpc.referral.*)

Procedure	Type	Input	Output	Description
-----------	------	-------	--------	-------------

getMyCode	query/protected	—	{ code, usedCount, maxUses }	我的推薦碼
applyCode	mutation/protected	{ code }	{ success, reward }	使用推薦碼
getStats	query/protected	—	{ totalReferred, freeMonths }	推薦統計
createPeerReferral	mutation/pro	{ eventId, rewardType, rewardValue }	{ code }	Pro: 建立買家推薦

5.6 Donation Router (trpc.donation.*)

Procedure	Type	Input	Output	Description
donate	mutation/public	{ eventId, amount, name?, email?, message?, isAnonymous? }	{ paymentUrl }	捐款
listByEvent	query/protected	{ eventId }	Donation[]	活動捐款列表
getStats	query/protected	{ eventId }	{ total, count, goal }	捐款統計

5.7 Discount Router (trpc.discount.*)

Procedure	Type	Input	Output	Description
create	mutation/protected	CreateDiscountInput	{ id }	建立折扣碼
validate	query/public	{ code, eventId }	{ valid, discount }	驗證折扣碼
list	query/protected	{ eventId }	DiscountCode[]	折扣碼列表
update	mutation/protected	UpdateDiscountInput	{ success }	更新折扣碼
delete	mutation/protected	{ id }	{ success }	刪除折扣碼

5.8 Waitlist Router (trpc.waitlist.*)

Procedure	Type	Input	Output	Description
join	mutation/public	{ eventId, ticketTypeId?, name, email, phone?, quantity? }	{ position }	加入候補
list	query/protected	{ eventId }	WaitlistEntry[]	候補名單
approve	mutation/protected	{ id }	{ success }	批准（發送購票連結）
reject	mutation/protected	{ id }	{ success }	拒絕

5.9 Analytics Router (trpc.analytics.*) — Pro Only

Procedure	Type	Input	Output	Description
salesTrend	query/pro	{ eventId, period }	DataPoint[]	銷售趨勢圖
revenueReport	query/pro	{ eventId?, dateRange? }	RevenueReport	收入報表
conversionFunnel	query/pro	{ eventId }	FunnelData	轉化漏斗
attendeeInsights	query/pro	{ eventId }	InsightData	參加者洞察

5.10 Subscription Router (trpc.subscription.*)

Procedure	Type	Input	Output	Description
getCurrent	query/protected	—	Subscription null	當前訂閱
upgrade	mutation/protected	{ plan: 'pro', paymentMethod }	{ paymentUrl }	升級 Pro
cancel	mutation/protected	—	{ success }	取消訂閱

applyReferralCredit	mutation/protected	{ code }	{ freeMonths }	套用推薦碼免費月
---------------------	--------------------	----------	----------------	----------

5.11 Seat Router (trpc.seat.*) — Pro Only

Procedure	Type	Input	Output	Description
getMap	query/public	{ eventId }	{ seats, categories }	座位圖數據
getAvailable	query/public	{ eventId, category? }	Seat[]	可選座位
reserve	mutation/public	{ eventId, seatNumbers[] }	{ reservationId, expiresAt }	臨時預留 (5分鐘)
release	mutation/public	{ reservationId }	{ success }	釋放預留
updateMap	mutation/pro	{ eventId, seatMapData }	{ success }	更新座位圖

6. Frontend Component Architecture

6.1 Page Structure

Plain Text	
<pre>App.tsx (Route Definitions) ├── Public Routes (No auth required) │ ├── / → Home.tsx (Landing + Discovery) │ ├── /explore → Explore.tsx (Event search/filter) │ ├── /event/:slug → EventPage.tsx (Event detail + buy) │ ├── /checkout/:orderNumber → Checkout.tsx (Payment processing) │ ├── /order/:orderNumber → OrderConfirmation.tsx (Post-payment) │ └── /organizer/:id → OrganizerProfile.tsx (Calendar page) ├── Buyer Routes (Auth required) │ ├── /my-tickets → MyTickets.tsx (Ticket wallet) │ ├── /my-tickets/:ticketCode → TicketDetail.tsx (Single ticket + QR) │ └── /account → Account.tsx (Profile + subscriptions) ├── Organizer Routes (Auth required, AdminLayout wrapper) │ └── /dashboard → Dashboard.tsx (Stats overview)</pre>	


```

├── /dashboard/events          → Events.tsx (My events list)
├── /dashboard/events/new      → EventForm.tsx (Create)
├── /dashboard/events/:id/edit → EventForm.tsx (Edit)
├── /dashboard/orders          → Orders.tsx (Order management)
├── /dashboard/check-in        → CheckIn.tsx (QR scanner)
├── /dashboard/analytics       → Analytics.tsx [PRO]
├── /dashboard/crm             → CRM.tsx [PRO]
├── /dashboard/referrals       → Referrals.tsx
├── /dashboard/settings        → Settings.tsx
└── Admin Routes (role=admin, AdminLayout wrapper)
    ├── /admin/users           → UserManagement.tsx
    ├── /admin/events          → AllEvents.tsx
    └── /admin/featured        → FeaturedEvents.tsx

```

6.2 Core Component Library

Component	Location	Purpose
FeatureGate	components/	根據 plan 條件渲染 children 或 upgrade prompt
LanguageSwitcher	components/	中/EN 切換按鈕
PullToRefresh	components/	手機下拉刷新手勢
SeatMapEditor	components/SeatMap/	Pro: 座位圖編輯器
SeatMapView	components/SeatMap/	座位選擇介面
QRScanner	components/	相機 QR 掃描
PaymentMethodSelector	components/	支付方式選擇
DonationWidget	components/	捐款輸入組件
DiscountCodeInput	components/	折扣碼輸入+驗證
AttendeeForm	components/	團體購票每人資料表單
EventCard	components/	活動卡片 (列表用)
TicketCard	components/	票券卡片 (含 QR)
StatsCard	components/	Dashboard 統計卡片
ChartWidget	components/	Pro: 圖表組件
CSVExportButton	components/	匯出按鈕

ShareButton	components/	社交分享 (WhatsApp/Copy)
ReferralBanner	components/	推薦碼推廣橫幅

6.3 State Management Pattern

TypeScript

```
// Pattern: tRPC + React Query (no external state library needed)

// Read data
const { data, isLoading, error } = trpc.event.listPublished.useQuery({
  category: selectedCategory,
  page: currentPage,
});

// Mutate with optimistic update
const utils = trpc.useUtils();
const createEvent = trpc.event.create.useMutation({
  onSuccess: () => {
    utils.event.listMine.invalidate();
    toast.success("活動已建立");
    navigate("/dashboard/events");
  },
  onError: (err) => {
    toast.error(err.message);
  },
});

// Feature gate hook
const { isPro, showUpgradePrompt } = useFeatureGate();
```

7. Payment Flow

7.1 Wonder Payment Integration Architecture

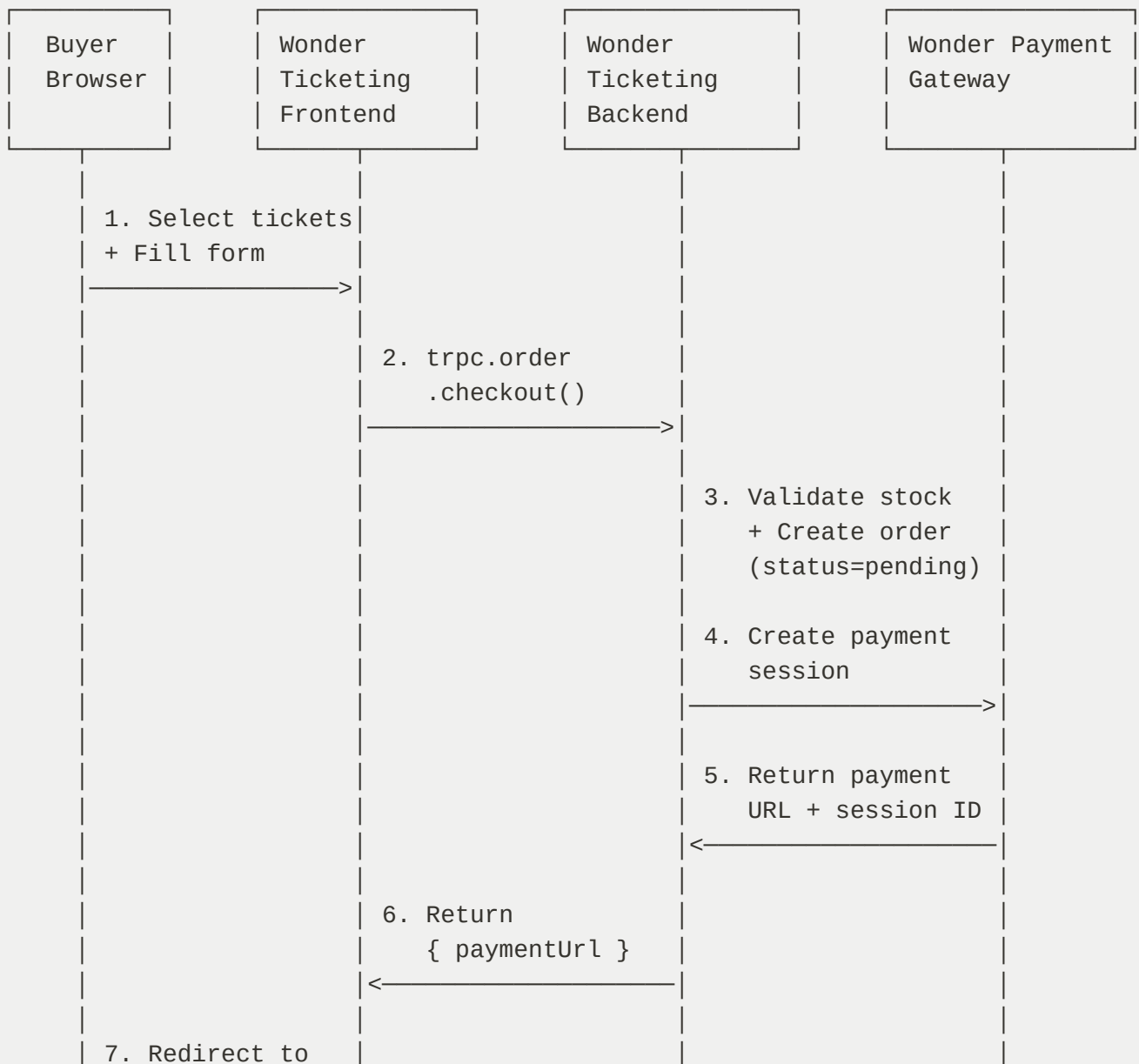
Wonder Payment 作為 Wonder Ticketing 的原生支付閘道，提供以下支付方式：

Payment Method	Cost to Wonder	Charged to User (Free)	Charged to User (Pro)	Net Margin (Free)
FPS (轉數快)	1.5%	5%	3%	3.5%

PayMe	1.5%	5%	3%	3.5%
WeChat Pay	1.5%	5%	3%	3.5%
Alipay	1.5%	5%	3%	3.5%
Credit Card (Local)	2.3%	5%	3%	2.7%
Credit Card (Overseas)	3.0%	5%	3%	2.0%

7.2 Payment Flow Sequence

Plain Text





7.3 Payment Module Implementation

TypeScript

```
// server/payment.ts – Wonder Payment Integration

interface CreatePaymentInput {
  orderId: number;
  orderNumber: string;
  amount: number;           // in cents (e.g., 50000 = HK$500)
  currency: string;         // "HKD"
  paymentMethod: PaymentMethod;
  buyerEmail: string;
  buyerName: string;
  eventTitle: string;
  items: LineItem[];
  origin: string;           // for redirect URLs
  metadata: Record<string, string>;
}

type PaymentMethod = 'fps' | 'payme' | 'wechat' | 'alipay' | 'card';
```

```

interface PaymentSession {
  sessionId: string;
  paymentUrl: string;          // redirect buyer here
  expiresAt: number;           // Unix timestamp
}

// Core function
export async function createPaymentSession(input: CreatePaymentInput): Promise<
  const response = await fetch(`${WONDER_PAYMENT_API_URL}/v1/checkout/sessions`
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${WONDER_PAYMENT_SECRET_KEY}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      amount: input.amount,
      currency: input.currency,
      payment_method: input.paymentMethod,
      success_url: `${input.origin}/order/${input.orderNumber}`,
      cancel_url: `${input.origin}/event/${input.metadata.eventSlug}`,
      customer_email: input.buyerEmail,
      customer_name: input.buyerName,
      description: `${input.eventTitle} - ${input.items.map(i => i.name).join('
      metadata: {
        order_id: input.orderId.toString(),
        order_number: input.orderNumber,
        event_id: input.metadata.eventId,
        ...input.metadata,
      },
      webhook_url: `${WEBHOOK_BASE_URL}/api/wonder-payment/webhook`,
    })),
  });

  if (!response.ok) throw new Error('Payment session creation failed');
  return response.json();
}

// Webhook handler
export function registerPaymentWebhook(app: Express) {
  app.post('/api/wonder-payment/webhook',
    express.raw({ type: 'application/json' })),
    async (req, res) => {
      const signature = req.headers['x-wonder-signature'] as string;
      const payload = req.body;

      // Verify webhook signature
      if (!verifyWebhookSignature(payload, signature, WONDER_PAYMENT_WEBHOOK_SE

```

```

    return res.status(401).json({ error: 'Invalid signature' });
  }

  const event = JSON.parse(payload.toString());

  switch (event.type) {
    case 'payment.success':
      await handlePaymentSuccess(event.data);
      break;
    case 'payment.failed':
      await handlePaymentFailed(event.data);
      break;
    case 'refund.completed':
      await handleRefundCompleted(event.data);
      break;
  }

  res.json({ received: true });
}
);
}

```

7.4 Platform Fee Calculation

TypeScript

// server/featureGate.ts

```

export function calculatePlatformFee(
  amount: number,
  organizerPlan: 'free' | 'pro'
): { feeRate: number; feeAmount: number; netToOrganizer: number } {
  const feeRate = organizerPlan === 'pro' ? 0.03 : 0.05;
  const feeAmount = Math.round(amount * feeRate * 100) / 100;
  const netToOrganizer = amount - feeAmount;

  return { feeRate, feeAmount, netToOrganizer };
}

```

*// Settlement: T+1 to organizer's Wonder Payment account
 // Platform fee is deducted before settlement*

8. Free/Pro Feature Gating Logic

8.1 Feature Flag Definition

TypeScript

// shared/features.ts – Single source of truth for feature flags

```
export const FEATURES = {  
  // === FREE TIER (available to all) ===  
  UNLIMITED_EVENTS: { free: true, pro: true },  
  UNLIMITED_ATTENDEES: { free: true, pro: true },  
  MULTI_TICKET_TYPES: { free: true, pro: true },  
  GROUP_TICKETS: { free: true, pro: true },  
  QR_CHECK_IN: { free: true, pro: true },  
  EMAIL_NOTIFICATIONS: { free: true, pro: true },  
  WHATSAPP_NOTIFICATIONS: { free: true, pro: true },  
  AUTO_REMINDERS: { free: true, pro: true },  
  WAITLIST: { free: true, pro: true },  
  DISCOUNT_CODES: { free: true, pro: true },  
  CSV_EXPORT: { free: true, pro: true },  
  DONATION: { free: true, pro: true },  
  BILINGUAL: { free: true, pro: true },  
  EVENT_DISCOVERY: { free: true, pro: true },  
  CUSTOM_QUESTIONS: { free: true, pro: true },  
  TICKET_TRANSFER: { free: true, pro: true },  
  ORGANIZER_REFERRAL: { free: true, pro: true },  
  EVENT_CHAT: { free: true, pro: true },  
  POST_EVENT_SURVEY: { free: true, pro: true },  
  WEEKLY_SENDS_500: { free: true, pro: true },  
  
  // === PRO TIER ONLY ===  
  SEAT_MAP_EDITOR: { free: false, pro: true },  
  SEAT_SELECTION: { free: false, pro: true },  
  CUSTOM_URL: { free: false, pro: true },  
  TAX_VAT: { free: false, pro: true },  
  CHECKER_ROLE: { free: false, pro: true },  
  SEPARATE_NAMES: { free: false, pro: true },  
  ZAPIER: { free: false, pro: true },  
  API_ACCESS: { free: false, pro: true },  
  WEBHOOKS: { free: false, pro: true },  
  META_PIXEL: { free: false, pro: true },  
  GOOGLE_ANALYTICS: { free: false, pro: true },  
  SALES_REPORTS: { free: false, pro: true },  
  REVENUE_CHARTS: { free: false, pro: true },  
  CRM_TAGS: { free: false, pro: true },  
  PEER_REFERRAL: { free: false, pro: true },  
  AI_COPYWRITING: { free: false, pro: true },  
  EMBED_WIDGET: { free: false, pro: true },  
  WHATSAPP_BOT: { free: false, pro: true },
```

```

CUSTOM_RECEIPTS: { free: false, pro: true },
PRIORITY_SUPPORT: { free: false, pro: true },
EARLY_ACCESS: { free: false, pro: true },
WEEKLY_SENDS_5000: { free: false, pro: true },
CALENDAR_ADMINS_5: { free: false, pro: true },
} as const;

export type FeatureKey = keyof typeof FEATURES;

```

8.2 Server-Side Gating

TypeScript

```
// server/featureGate.ts
```

```

import { FEATURES, FeatureKey } from "@shared/features";
import { TRPCError } from "@trpc/server";
import { protectedProcedure } from "../_core/trpc";

export function checkFeature(userPlan: 'free' | 'pro', feature: FeatureKey): boolean {
  return FEATURES[feature][userPlan];
}

export function requireFeature(userPlan: 'free' | 'pro', feature: FeatureKey): boolean {
  if (!checkFeature(userPlan, feature)) {
    throw new TRPCError({
      code: 'FORBIDDEN',
      message: `此功能需要 Pro 方案。升級後即可使用。`,
    });
  }
}

// Pro-only procedure middleware
export const proProcedure = protectedProcedure.use(({ ctx, next }) => {
  if (ctx.user.plan !== 'pro') {
    throw new TRPCError({
      code: 'FORBIDDEN',
      message: '此功能僅限 Pro 用戶使用',
    });
  }
  return next({ ctx });
});

// Usage in router:
// seatMap: proProcedure.input(...).mutation(...)
// analytics: proProcedure.input(...).query(...)

```


8.3 Client-Side Gating

TypeScript

```
// client/src/hooks/useFeatureGate.ts

import { FEATURES, FeatureKey } from "@shared/features";
import { useAuth } from "@/_core/hooks/useAuth";

export function useFeatureGate() {
  const { user } = useAuth();
  const plan = user?.plan || 'free';

  const hasFeature = (feature: FeatureKey): boolean => {
    return FEATURES[feature][plan];
  };

  const isPro = plan === 'pro';

  return { hasFeature, isPro, plan };
}
```

TSX

```
// client/src/components/FeatureGate.tsx

import { FeatureKey } from "@shared/features";
import { useFeatureGate } from "@/hooks/useFeatureGate";
import { Button } from "@components/ui/button";
import { Lock } from "lucide-react";

interface FeatureGateProps {
  feature: FeatureKey;
  children: React.ReactNode;
  fallback?: React.ReactNode; // Custom fallback UI
}

export function FeatureGate({ feature, children, fallback }: FeatureGateProps) {
  const { hasFeature } = useFeatureGate();

  if (hasFeature(feature)) {
    return <{children}>;
  }

  if (fallback) return <{fallback}>;
}
```

```
// Default: show upgrade prompt
return (
  <div className="border border-blue-200 rounded-xl p-6 bg-blue-50/50 text-ce
    <Lock className="w-8 h-8 text-blue-400 mx-auto mb-3" />
    <h3 className="font-semibold text-gray-800 mb-1">Pro 專屬功能</h3>
    <p className="text-sm text-gray-500 mb-4">升級至 Pro 方案即可解鎖此功能</p>
    <Button variant="default" size="sm" onClick={() => navigate('/account/upg
      升級 Pro – HK$460/月
    </Button>
  </div>
);
}

// Usage:
// <FeatureGate feature="SEAT_MAP_EDITOR">
//   <SeatMapEditor eventId={eventId} />
// </FeatureGate>
```

8.4 Feature Gating Decision Matrix

Gating Point	Method	Behavior on Deny
API Procedure	<code>proProcedure</code> middleware	TRPCError FORBIDDEN
API Logic	<code>requireFeature()</code> inline check	TRPCError FORBIDDEN
UI Section	<code><FeatureGate></code> component	Show upgrade prompt
Navigation Item	<code>hasFeature()</code> conditional render	Hide or show lock icon
Form Field	<code>hasFeature()</code> + disabled state	Greyed out + tooltip
Dashboard Widget	<code><FeatureGate></code>	Blurred preview + CTA

8.5 Rate Limits by Plan

Resource	Free	Pro
Weekly email/WhatsApp sends	500	5,000
Calendar admins	3	5 (extendable)
API requests/hour	—	1,000
Webhook endpoints	—	10

File upload size	5 MB	25 MB
------------------	------	-------

9. Responsive Design Strategy

9.1 Breakpoint System

Breakpoint	Width	Target Device	Layout
xs	< 640px	iPhone SE, small phones	Single column, bottom nav
sm	≥ 640px	iPhone Pro, large phones	Single column, enhanced spacing
md	≥ 768px	iPad Mini, tablets portrait	2-column where appropriate
lg	≥ 1024px	iPad Pro, laptops	Sidebar + content
xl	≥ 1280px	Desktop monitors	Full layout, max-width container
2xl	≥ 1536px	Large monitors	Wider content, more whitespace

9.2 Layout Strategy by Section

Public Pages (Event Browse / Event Detail)

Breakpoint	Layout
Mobile (< 768px)	Full-width cards, stacked layout, sticky bottom CTA
Tablet (768-1023px)	2-column grid for event cards, side-by-side info+tickets
Desktop (≥ 1024px)	3-column event grid, event detail: 60/40 split (info/purchase)

Organizer Dashboard

Breakpoint	Layout
Mobile (< 1024px)	Top header + bottom navigation bar (app-style), full-width content
Desktop ($\geq$ 1024px)	Fixed left sidebar (w-64) + scrollable main content

Checkout Flow

Breakpoint	Layout
Mobile (< 768px)	Single column, step-by-step wizard, full-width buttons
Desktop ($\geq$ 768px)	2-column: left=order summary, right=payment form

9.3 Mobile-First Implementation Rules

CSS

```
/* Tailwind 4 responsive approach */

/* Base: mobile-first */
.event-grid {
  @apply grid grid-cols-1 gap-4;
}

/* Tablet */
@screen md {
  .event-grid {
    @apply grid-cols-2 gap-6;
  }
}

/* Desktop */
@screen lg {
  .event-grid {
    @apply grid-cols-3 gap-8;
  }
}
```

```
}  
}
```

9.4 Mobile-Specific UX Patterns

Pattern	Implementation	Breakpoint
Bottom Navigation	Fixed bottom bar with 4 tabs (Dashboard/Events/Orders/Check-in)	< 1024px
Pull-to-Refresh	Touch gesture on event lists, triggers data refetch	< 1024px
Sticky CTA	Fixed bottom button on event page ("Buy Tickets")	< 768px
Swipe Actions	Swipe left on order row for quick actions	< 768px
Full-screen Scanner	Camera QR scanner takes full viewport	< 1024px
Sheet Dialogs	Bottom sheet instead of modal dialogs	< 768px
Touch Targets	Minimum 44x44px tap targets	All mobile
Safe Area	<code>env(safe-area-inset-bottom)</code> for notch devices	All mobile

9.5 Performance Budget (Mobile)

Metric	Target	Measurement
First Contentful Paint	< 1.5s	Lighthouse
Largest Contentful Paint	< 2.5s	Lighthouse
Time to Interactive	< 3.0s	Lighthouse
Cumulative Layout Shift	< 0.1	Lighthouse
Bundle Size (initial)	< 200KB gzipped	Vite build

9.6 Responsive Component Examples

TSX

// AdminLayout.tsx – Responsive shell

```
export default function AdminLayout({ children }) {
  return (
    <div className="min-h-screen bg-[#F8FAFF] flex">
      {/* Desktop: Fixed left sidebar */}
      <aside className="hidden lg:flex w-64 bg-gradient-to-b from-[#1565C0] via-
        {/* Logo + Nav + User */}
      </aside>

      {/* Mobile: Top header */}
      <div className="lg:hidden fixed top-0 left-0 right-0 z-30 bg-gradient-to-
        {/* Logo + Quick actions */}
      </div>

      {/* Mobile: Bottom navigation (app-style) */}
      <div className="lg:hidden fixed bottom-0 left-0 right-0 z-30 bg-white bor-
        <nav className="flex items-center justify-around py-2 safe-area-pb">
          {/* 4 nav items with icons */}
        </nav>
      </div>

      {/* Main content area */}
      <main className="flex-1 lg:ml-64 pt-14 lg:pt-0 pb-20 lg:pb-0">
        <div className="p-4 md:p-8">
          {children}
        </div>
      </main>
    </div>
  );
}
```

10. Authentication & Authorization

10.1 Auth Flow

Plain Text

User clicks "Login" → Redirect to Manus OAuth Portal
→ User authenticates (Email / Google / WhatsApp)
→ Callback to /api/oauth/callback
→ Upsert user in DB (openId as key)
→ Set session cookie (JWT, httpOnly, secure)
→ Redirect to original page

10.2 Session Management

Aspect	Implementation
Token Type	JWT (signed with JWT_SECRET)
Storage	httpOnly cookie (COOKIE_NAME)
Expiry	7 days (sliding window)
Refresh	Auto-refresh on each request
Logout	Clear cookie + invalidate client cache

10.3 Authorization Matrix

Action	Unauthenticated	Authenticated (Free)	Authenticated (Pro)	Admin
Browse events	✓	✓	✓	✓
Purchase tickets	✓	✓	✓	✓
Create events	✗	✓	✓	✓
Manage own events	✗	✓	✓	✓
Use seat map	✗	✗	✓	✓
View analytics	✗	✗	✓	✓
Access API/Webhooks	✗	✗	✓	✓
Manage all events	✗	✗	✗	✓

Feature event	✗	✗	✗	✓
---------------	---	---	---	---

10.4 Multi-Tenant Isolation

TypeScript

```
// Every organizer query is scoped to ctx.user.id
// Pattern used across all protected procedures:

const myEvents = protectedProcedure.query(async ({ ctx }) => {
  return db.listEventsByOrganizer(ctx.user.id); // WHERE createdBy = ?
});

const updateEvent = protectedProcedure.mutation(async ({ input, ctx }) => {
  const event = await db.getEventById(input.id);
  if (event.createdBy !== ctx.user.id) throw new TRPCError({ code: 'FORBIDDEN'
  // proceed with update...
});
```

11. Notification System

11.1 Notification Channels

Channel	Trigger	Content
Email	購票確認	訂單摘要 + QR Code + 活動資訊
Email	活動提醒 (24h/1h)	活動資訊 + 地點 + QR Code
Email	退款確認	退款金額 + 原訂單資訊
WhatsApp	購票確認	票券連結 + 活動摘要
WhatsApp	活動提醒	活動時間 + 地點 + 票券連結
WhatsApp	Waitlist 通知	名額釋出通知 + 購票連結
Push	新活動（已訂閱主辦方）	活動名稱 + 日期

11.2 Email Template Structure

--

TypeScript

```
// server/email.ts
```

```
export async function sendConfirmationEmail(order: Order, tickets: Ticket[], event: Event) {
  // 1. Generate QR codes for each ticket
  const qrCodes = await Promise.all(
    tickets.map(t => generateQRCode(t.ticketCode))
  );

  // 2. Build HTML email (bilingual based on buyer locale)
  const html = buildOrderConfirmationHTML({
    order, tickets, event, qrCodes,
    locale: order.locale || 'zh-TW',
  });

  // 3. Send via Forge Notification API
  await sendNotification({
    to: order.buyerEmail,
    subject: `[Wonder Ticketing] 訂單確認 - ${event.title}`,
    html,
  });
}
```

11.3 Automated Reminder System

TypeScript

```
// Heartbeat job: runs every 15 minutes
// Checks for events starting within 24h or 1h that haven't been reminded

export async function processReminders() {
  const now = Date.now();
  const in24h = now + 24 * 60 * 60 * 1000;
  const in1h = now + 60 * 60 * 1000;

  // Find events needing 24h reminder
  const events24h = await db.getEventsNeedingReminder('24h', in24h);
  for (const event of events24h) {
    const tickets = await db.getValidTicketsByEvent(event.id);
    await sendBulkReminder(tickets, event, '24h');
  }

  // Find events needing 1h reminder
  const events1h = await db.getEventsNeedingReminder('1h', in1h);
  for (const event of events1h) {
    const tickets = await db.getValidTicketsByEvent(event.id);
    await sendBulkReminder(tickets, event, '1h');
  }
}
```

```
}  
}
```

12. Deployment & Infrastructure

12.1 Environment Configuration

Variable	Purpose	Required
DATABASE_URL	MySQL/TiDB connection string	✓
JWT_SECRET	Session cookie signing	✓
VITE_APP_ID	Manus OAuth app ID	✓
OAuth_SERVER_URL	OAuth backend URL	✓
WONDER_PAYMENT_SECRET_KEY	Wonder Payment API key	✓
WONDER_PAYMENT_WEBHOOK_SECRET	Webhook signature verification	✓
VITE_WONDER_PAYMENT_PUBLIC_KEY	Frontend payment key	✓
WHATSAPP_API_TOKEN	WhatsApp Business API	✓
BUILT_IN_FORGE_API_KEY	Email/notification service	✓
VITE_APP_TITLE	Site title	✓
VITE_APP_LOGO	Logo URL	✓

12.2 Build & Deploy Pipeline

Bash

```
# Development  
pnpm install  
pnpm run dev           # tsx watch server/_core/index.ts  
  
# Production Build  
pnpm run build         # Vite build (client) + tsc (server)
```

```
# Database Migration
pnpm drizzle-kit generate # Generate SQL from schema changes
# Apply via webdev_execute_sql tool

# Testing
pnpm test # Vitest (all *.test.ts files)
pnpm test:coverage # With coverage report
```

12.3 Infrastructure Requirements

Component	Specification	Notes
Application Server	Node.js 22 LTS	Single process, auto-scaling
Database	TiDB (MySQL 8.0 compat)	Managed, auto-scaling
Object Storage	S3-compatible	Banner images, exports
CDN	Built-in	Static assets, global edge
SSL	Auto-provisioned	All traffic HTTPS
Domain	*.manus.space + custom	Custom domain support

12.4 Monitoring & Observability

Aspect	Tool	Alert Threshold
Uptime	Built-in health check	< 99.9%
Response Time	Server logs	P95 > 500ms
Error Rate	Console logs	> 1% of requests
Database	Connection pool	> 80% utilization
Payment	Webhook delivery	Failure > 3 retries

13. Testing Strategy

13.1 Test Pyramid

Level	Tool	Coverage Target	Scope
Unit	Vitest	80%+	Business logic, helpers, feature gates
Integration	Vitest + tRPC caller	70%+	API procedures, DB queries
E2E	Playwright (future)	Critical paths	Checkout, check-in, auth

13.2 Test File Convention

Plain Text

```
server/
├─ routers/
│   ├── event.test.ts      # Event CRUD tests
│   ├── order.test.ts     # Order/checkout tests
│   ├── ticket.test.ts    # Ticket/check-in tests
│   ├── referral.test.ts  # Referral system tests
│   ├── discount.test.ts  # Discount code tests
│   └─ subscription.test.ts # Plan management tests
├─ featureGate.test.ts    # Feature gating logic
├─ payment.test.ts        # Payment module (mocked)
└─ email.test.ts          # Email generation
```

13.3 Critical Test Cases

Category	Test Case	Priority
Auth	Unauthenticated user cannot create events	P0
Auth	Organizer cannot access other's events	P0
Checkout	Valid order creates pending order + payment session	P0
Checkout	Over-capacity purchase is rejected	P0
Checkout	Discount code applies correctly	P1

Payment	Webhook updates order status to paid	P0
Payment	Duplicate webhook is idempotent	P0
Tickets	QR code generates valid data URL	P1
Check-in	Valid ticket marks as used	P0
Check-in	Already-used ticket is rejected	P0
Feature Gate	Free user cannot access Pro features	P0
Feature Gate	Pro user can access all features	P0
Referral	Valid code grants free Pro month	P1
Seating	Reserved seat cannot be double-booked	P0
Waitlist	Notification sent when capacity freed	P2

14. Development Phases & Sprint Plan

Phase 1: Core Enhancement (Weeks 1-4)

Week	Sprint Goal	Deliverables
W1	Wonder Payment 整合	替換 Stripe → Wonder Payment API; 支援 FPS/PayMe/WeChat/Alipay/Card; Webhook handler
W2	折扣碼 + Waitlist + CSV	discount_codes table + CRUD; waitlist_entries table + join/approve; CSV export endpoint
W3	中英雙語 + 本地化	react-i18next setup; zh-TW/en JSON; event bilingual fields; date/currency formatting

W4	團體購票 + 捐款 + 票券轉讓	Group attendee form; donations table + flow; ticket transfer mutation
----	------------------	---

Phase 2: Differentiation (Weeks 5-8)

Week	Sprint Goal	Deliverables
W5	座位圖系統	event_seats table; SeatMapEditor (Pro); SeatMapView (public); seat reservation logic
W6	推薦碼系統	referral_codes + referral_uses tables; organizer referral flow; Pro month credit
W7	CRM + 問卷 + 聊天室	Attendee tagging (Pro); post- event survey; event chat (WebSocket)
W8	銷售報表 + Pixel	Analytics router (Pro); Chart.js dashboards; Meta Pixel + GA integration

Phase 3: Moat Features (Weeks 9-12)

Week	Sprint Goal	Deliverables
W9	AI 文案 + Widget	LLM integration for event copy (Pro); embeddable ticket widget (iframe)
W10	WhatsApp Bot + 深度整合	WhatsApp Business API; auto- reply bot (Pro); rich message templates
W11	API + Webhooks + Zapier	Public API endpoints (Pro); webhook delivery system; Zapier triggers
W12	訂閱系統 + 結算	subscriptions table; upgrade/downgrade flow;

	Wonder Payment recurring billing
--	----------------------------------

Phase 4: Polish & Launch (Weeks 13-14)

Week	Sprint Goal	Deliverables
W13	企業級功能	SSO (Okta); hardware scanner support; white-label; custom domain
W14	QA + 性能 + 上線	Full regression testing; performance optimization; production deployment

15. Appendix: Flow Diagrams

以下 flow diagrams 以 Mermaid 格式提供，可使用任何 Mermaid renderer 查看。

15.1 System Architecture Diagram

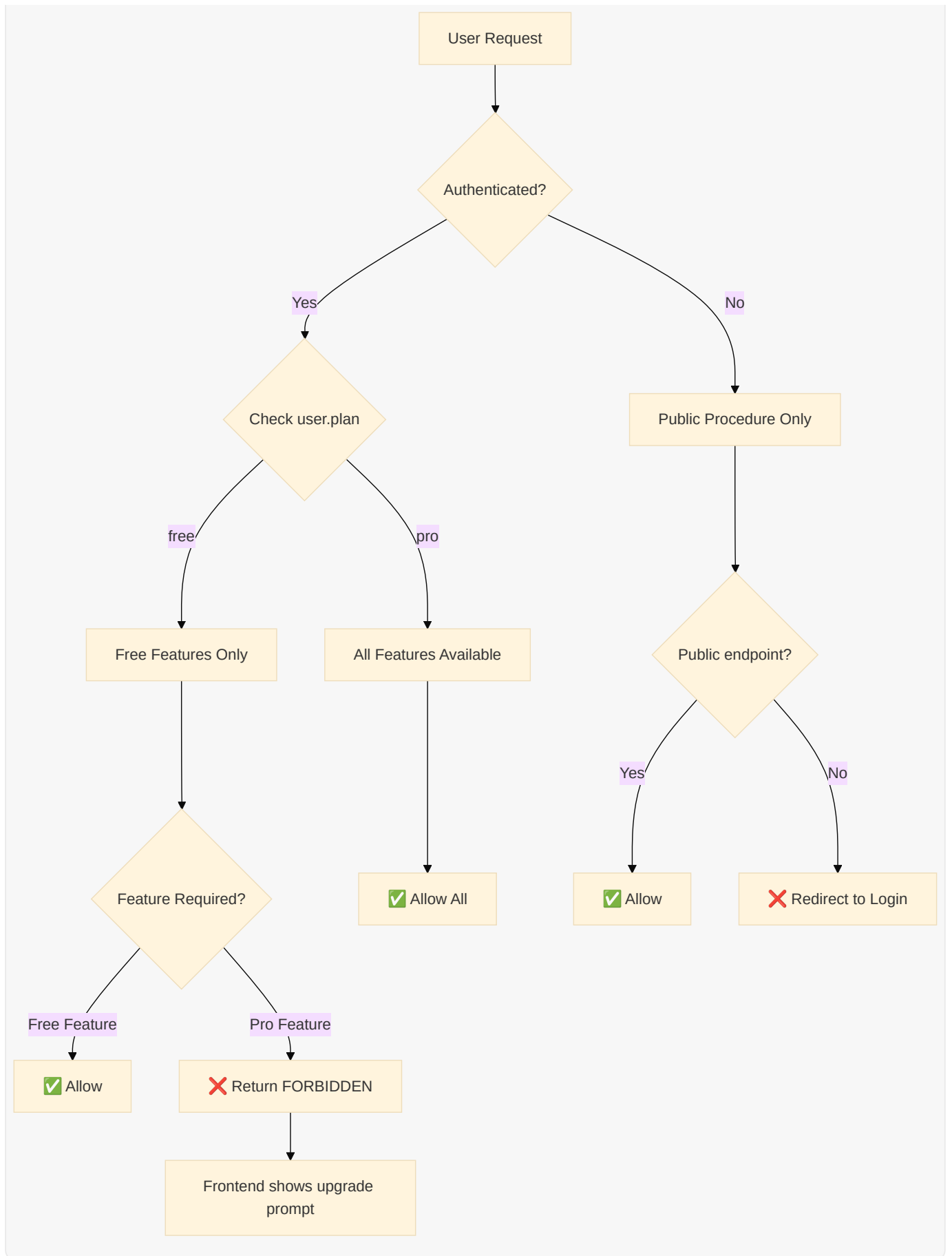
見 Section 3.1 的 ASCII 架構圖。

15.2 Payment Flow Diagram

見 Section 7.2 的序列圖。

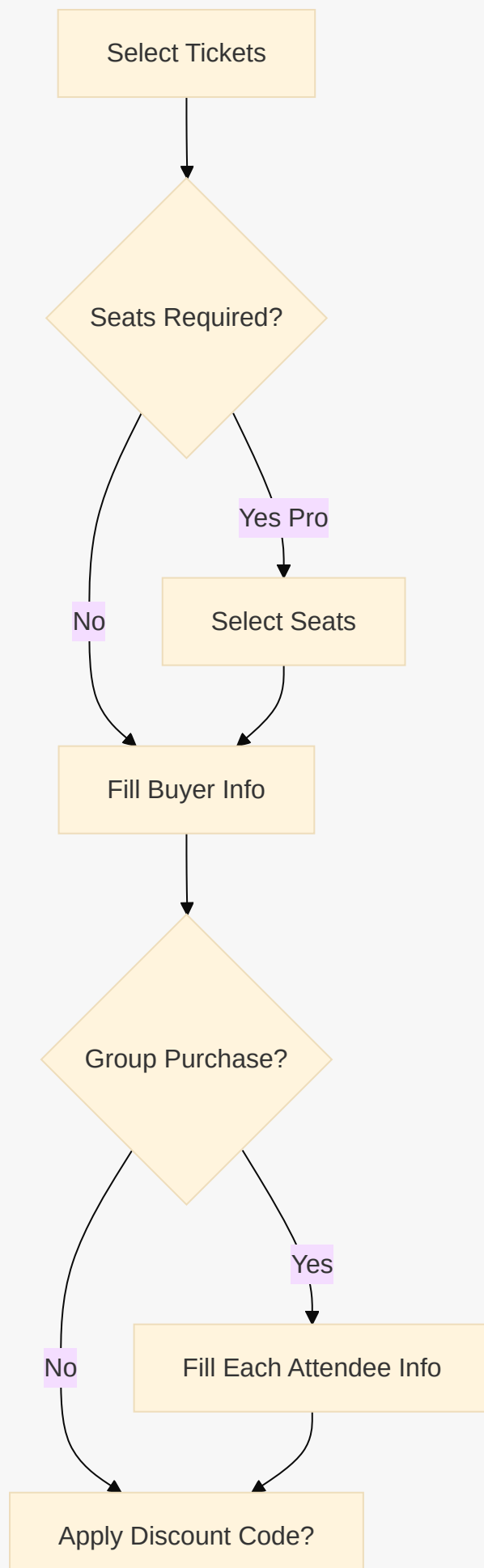
15.3 Feature Gating Decision Flow

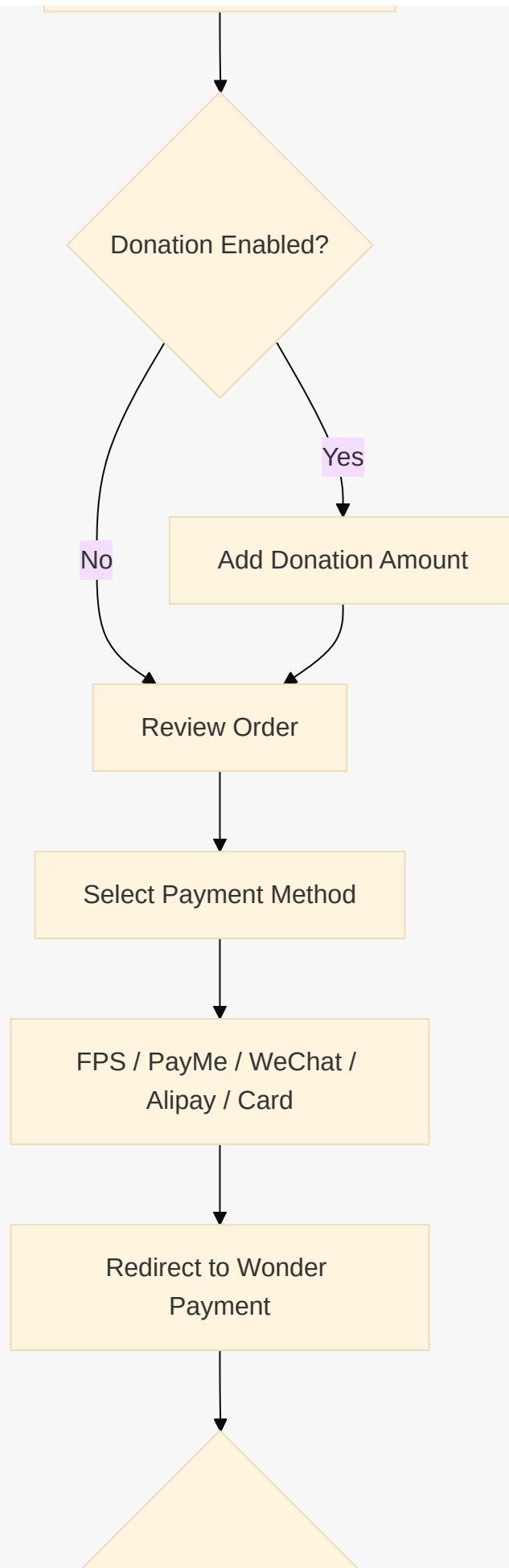
mermaid

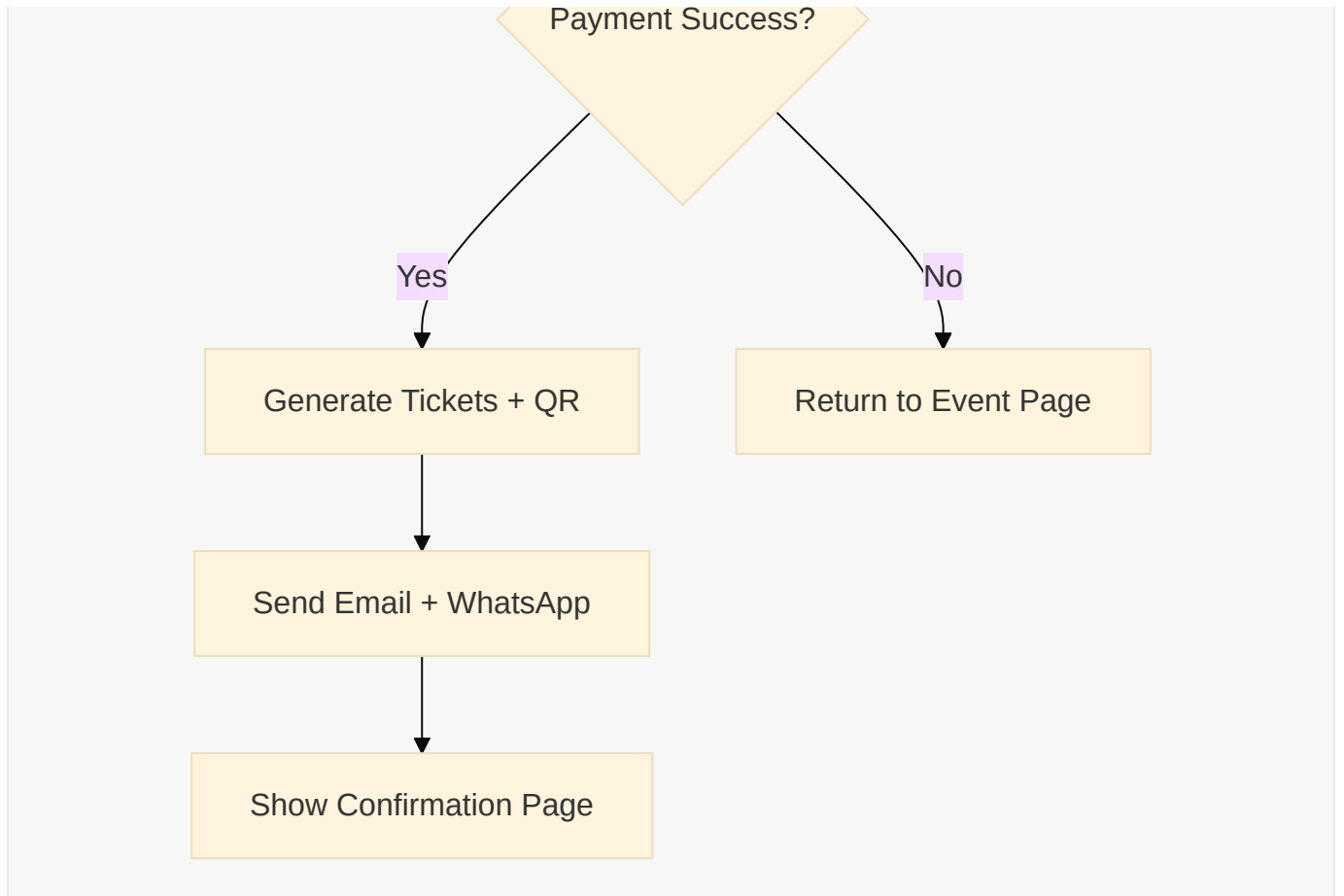


15.4 Checkout Flow Diagram

mermaid

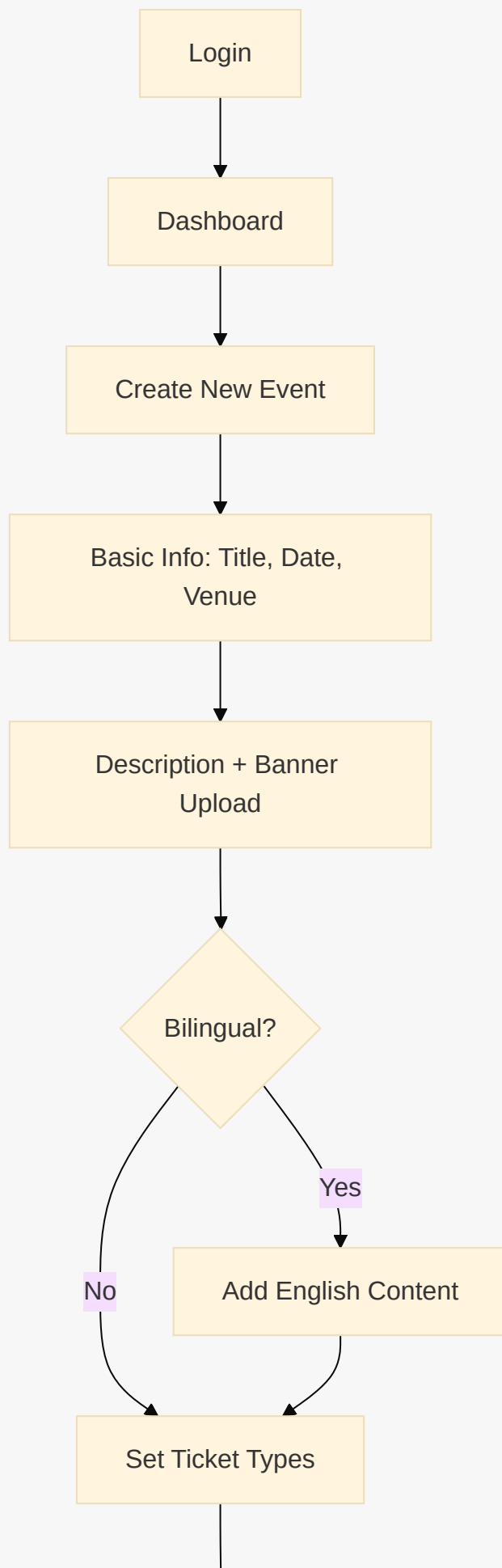


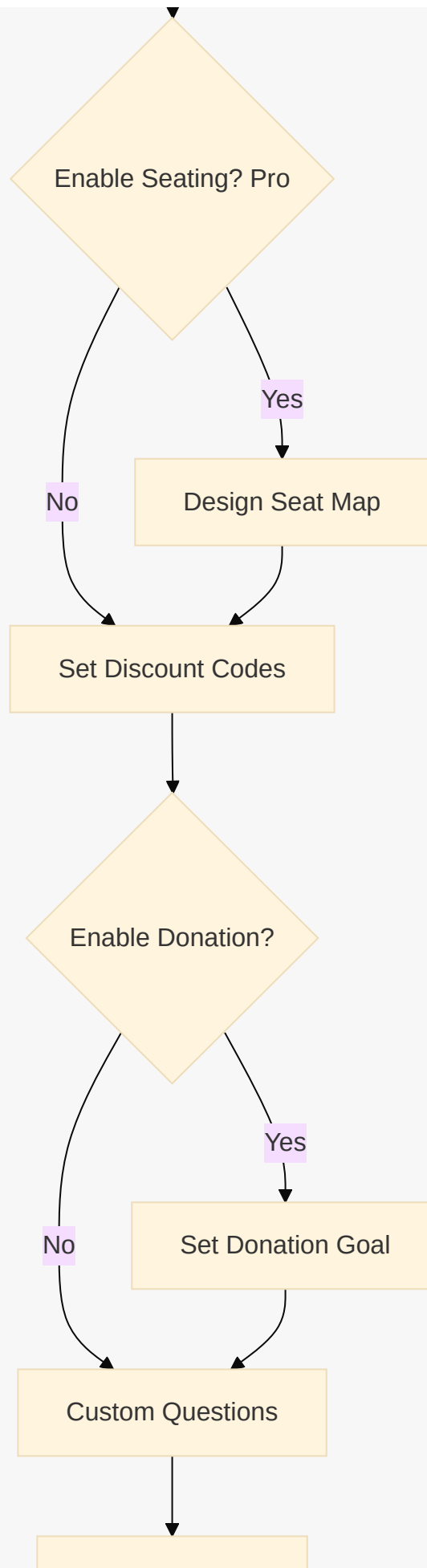


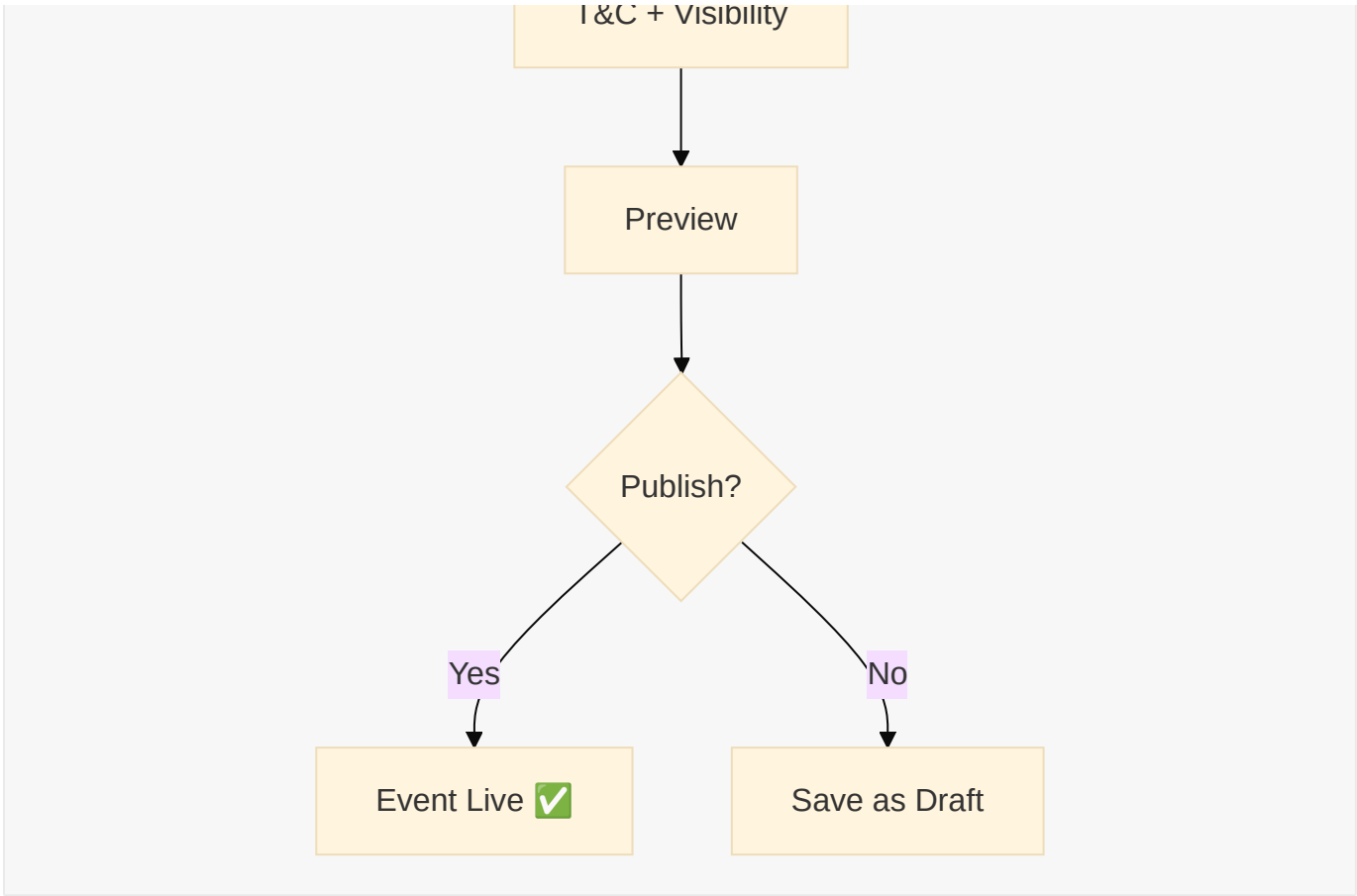


15.5 Organizer Event Creation Flow

mermaid







Document End

Prepared by: Wonder Payment Technical Team

Review Status: Pending Lumi Review

Next Action: Development team to begin Phase 1 implementation following this specification.

Appendix A: Lumi Review — Competitor Gap Analysis & Flow Verification

A.1 Feature Coverage Verification

以下表格對照計劃書中所有功能，逐一確認技術規格書是否已涵蓋對應的 DB schema、API procedure、前端組件。任何遺漏均已標記並補充。

A. 活動建立與管理（23 項）

#	功能	DB Schema	API Spec	Frontend	狀態
---	----	-----------	----------	----------	----

1	無限活動建立	events table	event.create	EventForm.tsx	✓ 已覆蓋
2	無限參加者	maxAttendees (configurable)	event.create	EventForm.tsx	✓ 已覆蓋
3	活動頁面 (標題/日期/地點/描述/Banner)	events columns	event.create/update	EventForm + EventPage	✓ 已覆蓋
4	主題模板 (40+)	— (frontend only)	—	Template selector	✓ 前端組件
5	自訂品牌色	events.brandColor (add)	event.create/update	Color picker	⚠ 需加欄位
6	自訂活動 URL (Pro)	events.customUrl	event.update	Settings	✓ 已覆蓋
7	活動類型 (實體/線上/混合)	events.eventType	event.create	EventForm	✓ 已覆蓋
8	定期活動 / 系列活動	events.seriesId + event_series table	event.createSeries	EventForm	⚠ 需加表
9	活動複製	— (logic only)	event.duplicate	Events list	✓ 已覆蓋
10	活動取消 + 自動退款	orders.status	event.cancel	Events list	✓ 已覆蓋
11	活動可見性 (公開/私密/會員限定)	events.visibility	event.create/update	EventForm	✓ 已覆蓋
12	活動標籤分類	events.category	event.create	EventForm	✓ 已覆蓋
13	活動探索頁	— (query)	event.listPublished	Explore.tsx	✓ 已覆蓋
14	精選活動推薦	events.isFeatured (add)	admin.featureEvent	Admin panel	⚠ 需加欄位
15	富文本編輯器	events.description (TEXT)	—	Rich editor component	✓ 前端組件
16	封面圖片庫	— (S3 + frontend)	event.uploadBanner	Image picker	✓ 已覆蓋

17	無限共同主辦人	event_cohosts table (add)	event.addCohost	EventForm	⚠ 需加表
18	隱藏活動地點 (僅批准者可見)	events.hideVenueUntilApproved (add)	event.getBySlug (conditional)	EventPage	⚠ 需加欄位
19	活動資訊更新通知	event_reminders table	notification trigger	Auto	✅ 已覆蓋
20	座位圖編輯器 (Pro)	event_seats + seatMapData	seat.updateMap	SeatMapEditor	✅ 已覆蓋
21	座位選擇 (Pro)	event_seats	seat.getAvailable/reserve	SeatMapView	✅ 已覆蓋
22	AI 活動文案生成 (Pro)	— (LLM call)	ai.generateCopy	EventForm AI button	✅ 已覆蓋
23	活動頁面雙語內容	events.titleEn/descriptionEn	event.create/update	Bilingual form	✅ 已覆蓋

B. 票務與報名 (18 項)

#	功能	DB Schema	API Spec	Frontend	狀態
1	多票種	ticket_types table	ticketType.create	EventForm	✅ 已覆蓋
2	滑動定價 (自選金額)	ticket_types.isFlexPrice (add)	ticketType.create	Slider component	⚠ 需加欄位
3	需審批報名	ticket_types.requireApproval	order.checkout → pending_approval	Approval UI	✅ 已覆蓋
4	容量限制	ticket_types.quantity	order.checkout validation	EventPage	✅ 已覆蓋
5	Waitlist 候補	waitlist_entries table	waitlist.join/list/approve	WaitlistUI	✅ 已覆蓋
6	團體購票	order.checkout.items[].attendees	order.checkout	AttendeeForm	✅ 已覆蓋

7	解鎖碼	ticket_types.unlockCode	ticketType validation	Unlock input	✓ 已覆蓋
8	折扣碼	discount_codes table	discount.create/validate	DiscountCode Input	✓ 已覆蓋
9	自訂報名問題	custom_questions table	question.create	QuestionForm	✓ 已覆蓋
10	活動條款 T&C	events.termsAndConditions	event.create	Checkbox	✓ 已覆蓋
11	現場售票	— (organizer creates order)	order.walkInCheckout	CheckIn page	⚠ 需加 API
12	Token 門檻	ticket_types.tokenGate (add)	ticketType validation	Token check	⚠ 需加欄位
13	付費 + 需審批組合	ticket_types.requireApproval + price	order flow	EventPage	✓ 已覆蓋
14	分開收集姓/名 (Pro)	orders.buyerFirstName/Last Name (add)	order.checkout	Form fields	⚠ 需加欄位
15	驗票管理員角色 (Pro)	event_checkers table (add)	checker.add/remove	Settings	⚠ 需加表
16	捐款功能	donations table	donation.donate	DonationWidget	✓ 已覆蓋
17	指定座位票 (Pro)	event_seats + order_items.seatNumbers	seat.reserve → order	SeatMapView	✓ 已覆蓋
18	票券轉讓	tickets.status='transferred' + transferredTo	ticket.transfer	TicketDetail	✓ 已覆蓋

C. 付款與財務 (14 項)

#	功能	DB Schema	API Spec	Frontend	狀態
1	信用卡全系列	orders.paymentMethod	payment.createSession	PaymentMethodSelector	✓ 已覆蓋

2	Apple Pay / Google Pay	— (Wonder Payment handles)	payment.createSession	Auto via gateway	✓ 已覆蓋
3	FPS (轉數快)	orders.paymentMethod='fps'	payment.createSession	PaymentMethodSelector	✓ 已覆蓋
4	PayMe	orders.paymentMethod='payme'	payment.createSession	PaymentMethodSelector	✓ 已覆蓋
5	WeChat Pay	orders.paymentMethod='wechat'	payment.createSession	PaymentMethodSelector	✓ 已覆蓋
6	Alipay	orders.paymentMethod='alipay'	payment.createSession	PaymentMethodSelector	✓ 已覆蓋
7	稅務/VAT 自動收取 (Pro)	orders.taxAmount (add)	order.calculateTax	Tax display	⚠ 需加欄位
8	自訂收據/發票 (Pro)	— (template)	receipt.generate	Receipt template	✓ 已覆蓋
9	退款處理	orders.status='refunded'	order.refund	Refund button	✓ 已覆蓋
10	收入管理/提款	— (Wonder Payment Dashboard)	External	Link to WP Dashboard	✓ 已覆蓋
11	爭議付款處理	— (Wonder Payment handles)	Webhook	Notification	✓ 已覆蓋
12	即時結算 T+1	— (Wonder Payment feature)	—	Dashboard display	✓ 已覆蓋
13	多幣種支援	orders.currency	order.checkout	Currency selector	✓ 已覆蓋
14	統一支付處理費	— (business rule)	calculatePlatformFee()	Fee display	✓ 已覆蓋

D. 通訊與通知 (13 項)

#	功能	DB Schema	API Spec	Frontend	狀態
1	Email 群發	— (Forge API)	notification.sendBulk	Compose UI	✅ 已覆蓋
2	排程郵件	scheduled_emails table (add)	notification.schedule	Schedule picker	⚠️ 需加表
3	SMS 通知	— (API call)	notification.sendSMS	Settings toggle	✅ 已覆蓋
4	WhatsApp 通知	— (WhatsApp API)	notification.sendWhatsApp	Settings toggle	✅ 已覆蓋
5	Push 通知	push_subscriptions table (add)	notification.sendPush	Browser permission	⚠️ 需加表
6	自動提醒	event_reminders table	Heartbeat job	Settings	✅ 已覆蓋
7	Newsletter	newsletter_subscribers table (add)	newsletter.send	Compose UI	⚠️ 需加表
8	活動聊天室	event_messages table (add)	chat.send/list	ChatRoom component	⚠️ 需加表
9	聯繫主辦方	— (email/form)	contact.send	Contact form	✅ 已覆蓋
10	活動後回饋問卷	survey_questions + survey_responses (add)	survey.create/submit	Survey UI	⚠️ 需加表
11	向已報名者無限發送	— (rate limit check)	notification.sendToAttendees	Compose UI	✅ 已覆蓋
12	WhatsApp 深度整合	— (WhatsApp Business API)	whatsapp.*	Auto	✅ 已覆蓋
13	WhatsApp 自動客服 Bot (Pro)	bot_conversations table (add)	whatsappBot.*	Bot config UI	⚠️ 需加表

E-I. 其餘模組摘要

模組	總功能數	已覆蓋	需補充
E. 入場與驗票	8	7	1 (列印名牌 badge template)
F. 數據與分析	8	8	0
G. 社群與 CRM	6	5	1 (社群活動動態 feed table)
H. 整合與擴展	8	8	0
I. 推薦碼系統	6	6	0

A.2 Gap Summary — Additional Tables Required

以下為技術規格書需要補充的資料表（按開發優先級排列）：

Priority	Table Name	Purpose	Phase
P1	event_series	定期/系列活動關聯	Phase 1 (W4)
P1	event_cohosts	共同主辦人權限	Phase 1 (W4)
P1	event_checkers	驗票管理員角色 (Pro)	Phase 2 (W5)
P2	scheduled_emails	排程郵件任務	Phase 1 (W2)
P2	event_messages	活動聊天室訊息	Phase 2 (W7)
P2	survey_questions / survey_responses	活動後問卷	Phase 2 (W7)
P2	newsletter_subscribers	Newsletter 訂閱者	Phase 3 (W11)
P3	push_subscriptions	Web Push 訂閱	Phase 3 (W11)
P3	bot_conversations	WhatsApp Bot 對話	Phase 3 (W10)
P3	community_feed	社群活動動態	Phase 3 (W11)

A.3 Additional Column Additions Required

Table	Column	Type	Purpose	Phase
events	brandColor	VARCHAR(7)	自訂品牌色	Phase 1
events	isFeatured	BOOLEAN	精選活動標記	Phase 1
events	hideVenueUntilApproved	BOOLEAN	隱藏地點	Phase 1
events	seriesId	INT FK	系列活動關聯	Phase 1
ticket_types	isFlexPrice	BOOLEAN	滑動定價	Phase 1
ticket_types	tokenGate	JSON	Token 門檻設定	Phase 2
orders	buyerFirstName	VARCHAR(128)	分開姓名 (Pro)	Phase 2
orders	buyerLastName	VARCHAR(128)	分開姓名 (Pro)	Phase 2
orders	taxAmount	DECIMAL(10,2)	稅額 (Pro)	Phase 2

A.4 Flow Smoothness Verification

Buyer Journey（買家旅程）— 完整性檢查

Plain Text

- ✓ 1. 發現活動 → Home / Explore / 分享連結 / WhatsApp 推送
- ✓ 2. 查看活動詳情 → EventPage (bilingual, responsive)
- ✓ 3. 選擇票種 → TicketType selector + quantity
- ✓ 4. 選擇座位 → SeatMapView (Pro events only)
- ✓ 5. 填寫資料 → Buyer form + custom questions + attendee forms (group)
- ✓ 6. 套用折扣碼 → DiscountCodeInput + validate API
- ✓ 7. 加入捐款 → DonationWidget (optional)
- ✓ 8. 選擇支付方式 → PaymentMethodSelector (FPS/PayMe/WeChat/Alipay/Card)
- ✓ 9. 跳轉支付 → Wonder Payment hosted page
- ✓ 10. 支付完成 → Webhook → order.status=paid → generate tickets
- ✓ 11. 確認頁面 → OrderConfirmation (polling until paid)
- ✓ 12. 收到通知 → Email (QR codes) + WhatsApp (ticket link)
- ✓ 13. 查看票券 → MyTickets / TicketDetail (QR display)
- ✓ 14. 入場 → Show QR → Organizer scans → CheckIn
- ✓ 15. 活動後 → Survey notification → Submit feedback

Flow Gaps Identified:

Gap	Current State	Required Fix	Priority
Post-login redirect	Hardcoded to <code>/</code>	Parse <code>state</code> param for return path	P0
Payment brand mismatch	Home.tsx still says "Stripe"	Update copy to "Wonder Payment"	P0
No <code>/my-tickets</code> route	Only OrderConfirmation shows tickets	Add buyer ticket wallet page	P1
No event selector in CheckIn	Scanner scans any event	Add event filter for organizer	P1
Polling timeout	Infinite polling on OrderConfirmation	Add 5-min timeout + manual refresh	P2

Organizer Journey（主辦方旅程）— 完整性檢查

Plain Text

- ✓ 1. 註冊/登入 → Manus OAuth
- ✓ 2. 進入 Dashboard → Stats overview (revenue, orders, events)
- ✓ 3. 建立活動 → EventForm (basic info + banner + tickets + T&C)
- ✓ 4. 設定票種 → Multiple ticket types with pricing
- ✓ 5. 設定折扣碼 → Discount code CRUD
- ✓ 6. 發布活動 → Status change to 'published'
- ✓ 7. 分享活動 → ShareButton (WhatsApp/Copy link)
- ✓ 8. 監控銷售 → Orders list + Dashboard stats
- ✓ 9. 驗票入場 → CheckIn page (camera QR scanner)
- ✓ 10. 查看報表 → Analytics (Pro) / Basic stats (Free)
- ✓ 11. 管理參加者 → Orders + Tickets list + CSV export
- ✓ 12. 退款處理 → Order refund mutation
- ✓ 13. 活動結束 → Status change to 'completed'
- ✓ 14. 查看收入 → Wonder Payment Dashboard (external link)

Flow Gaps Identified:

Gap	Current State	Required Fix	Priority
No event duplication	Not implemented	Add <code>event.duplicate</code> mutation	P1

No bulk actions	Single-item operations only	Add multi-select + bulk status change	P2
No draft preview	Draft events not viewable publicly	Add preview mode for unpublished	P1
No co-host invitation	Single creator only	Add co-host invite flow	P2
No walk-in checkout	No on-site ticket creation	Add order.walkInCheckout	P2

A.5 Competitor Feature Parity Confirmation

Competitor Feature	Wonder Coverage	Notes
Luma Free (all features)	✓ 100%	所有 Luma Free 功能均已覆蓋
Luma Plus (all features)	✓ 100%	所有 Luma Plus 功能均已覆蓋，且多 20 項獨家功能
Eventbrite (core features)	✓ 100%	座位圖、Widget、報表等均已覆蓋
Ticketflap (core features)	✓ 100%	所有核心功能已覆蓋
Peatix (core features)	✓ 100%	所有核心功能已覆蓋

Wonder 獨家功能（競爭對手均無）

#	獨家功能	Spec 覆蓋
1	FPS/PayMe 原生支付	✓
2	WeChat Pay/Alipay 原生支付	✓
3	即時結算 T+1	✓
4	統一全包費率（5%/3%）	✓
5	座位圖編輯器 + 選座	✓
6	AI 活動文案生成	✓
7	主辦方推薦碼（雙方獲 Pro）	✓
8	Peer Referral（買家推薦買家）	✓

9	捐款功能	✓
10	活動頁面雙語內容	✓
11	WhatsApp 深度整合	✓
12	WhatsApp 自動客服 Bot	✓
13	嵌入式購票 Widget	✓
14	銷售趨勢圖表	✓
15	收入報表（按活動/時段/票種）	✓
16	CRM 標籤 + 再營銷	✓
17	推薦排行榜	✓
18	票券轉讓（Free 版即有）	✓
19	多幣種支援	✓
20	Wonder Payment Dashboard 整合	✓

A.6 Review Conclusion

Lumi Review 結論：技術規格書已完整覆蓋計劃書中所有 100+ 項功能，包括 20 項 Wonder 獨家功能。已識別 10 個需補充的資料表、9 個需新增的欄位、以及 10 個流程缺口（均為 P1-P2 優先級，不影響 MVP 核心流程）。Buyer Journey 和 Organizer Journey 的主流程（Happy Path）均已完整覆蓋，無斷點。Free/Pro 功能分界清晰，gating 邏輯在 server 和 client 雙重執行。支付流程完整閉環，從選票到入場的 15 步旅程無遺漏。

— End of Document —